than passed to STDOUT or Apache. This way you can accumulate output without triggering any side effects such as response headers or status lines. Let's see what happens:

```
// index.php

ob_start();
print "HTTP Status Code was " . http_response_code() . "\n";
print "But I don't like you so you are forbidden.\n";
http_response_code(403);

print "While we're at it let's sort that X- header out.\n";
header('X-Powered-By: none/of-your-business');
```

```
$ curl -v http://localhost:8080/index.php
< HTTP/1.1 403 Forbidden
< Host: localhost:8080
< Date: Mon, 14 Jul 2025 18:00:53 GMT
< Connection: close
< X-Powered-By: none/of-your-business
< Content-type: text/html; charset=UTF-8
<
HTTP Status Code was 200
But I don't like you so you are forbidden.
While we're at it let's sort that X- header out.
```

Now we're able to emit as much output as we like; the output buffer is catching everything, and that allows us to make changes to the status code and headers freely. What we didn't do, though, was clean up after ourselves nicely. The output buffer continued to accumulate data until the script exited, which triggered a flush of the buffer as a side effect. It's better practice to do it yourself with an explicit call either to `flush()` (outputs the contents of the buffer but does not disable the buffer) or `ob_end_flush()` (flushes the buffer and stops any further buffering).

Output buffering in PHP can get quite complex; for example, you can pass a `callable` to `ob_start()`, which will be called on each flush event. The contents of the buffer are passed to the callback when the buffer is flushed: